

Real-Time Fraud, Moderation, and Ad-Serving Architecture for Contextual Advertising

Adstract

1 Introduction and Problem Overview

We aim to build a real-time **advertising network** designed specifically for LLM-based platforms. Unlike traditional advertising systems (which rely on user profiles, browsing history, or personalization), our system identifies **advertisable contextual opportunities** based on the current conversation between a user and an LLM. Ads must be:

- contextually relevant to the user’s expressed need,
- ethically placed and clearly marked as sponsored,
- inserted naturally into the LLM response using instructions to minimize disruption,
- privacy-safe, with no user profiling.

Platforms integrate our SDK and send ad request. Upon receiving an ad request, the backend performs:

1. **Fraud and Moderation Pipeline** – ensuring the traffic is legitimate and the conversation is appropriate for advertising.
2. **Ad Retrieval and Selection** – identifying relevant ads using vector search and running an auction among candidate ads.
3. **Response Construction** – generating a *placement instruction* for the LLM and attaching any needed tracking links (for CPC models).
4. **Metrics Collection** – the SDK sends impression, click, and quality feedback for downstream analytics.

This document focuses on the **fraud, moderation and CTR pipeline**, and on how real-time data processing supports it by continuously updating fraud signals, moderation-related features, and CTR performance metrics used in downstream decision-making.

2 Fraud and Moderation Pipeline

Before serving any advertisement, the platform must guarantee:

1. The traffic is **authentic** and not artificially generated by malicious publishers.
2. The conversation context is **safe, appropriate, and suitable** for ad placement.

These two components—Fraud Detection and Content Moderation—form the first stage of the ad-serving pipeline.

2.1 Fraud Detection (Traffic Integrity)

Fraud detection aims to determine whether an ad request originates from:

- a legitimate user interacting with a real LLM platform, or
- a publisher attempting to generate fake inventory (e.g., bots simulating conversations to farm revenue).

Fraud signals include:

- abnormal request rates per publisher or IP,
- low-diversity IP/UA patterns,
- inconsistent geo distribution,
- unnatural click timing or excessively high CTR,
- repeated or nonsensical conversation patterns.

Because these patterns emerge continuously, fraud detection benefits strongly from **real-time streaming analytics**. Each event (ad request, ad response, click) contributes to a rolling feature set used to compute *fraud risk scores* for each publisher, platform, and source.

These risk scores are persisted in a fast store (e.g., Redis) and checked by the Django ad-serving API in the synchronous request path. If the score exceeds a threshold, the system refuses to serve an ad.

2.2 Content Moderation (Safety and Suitability)

Even if the request is legitimate, the conversation itself may be inappropriate for advertising. The system evaluates:

- sensitive or unsafe content (self-harm, explicit content, hateful language, medical harm, etc.),
- contextual suitability (e.g., a very short conversation, repeated messages, or unsuitable topics),

- ad fatigue (preventing ads from appearing too frequently in the same conversation).

Moderation includes:

1. **Fast per-request classification** using lightweight ML or rule-based safety models.
2. **Conversation-level heuristics**, such as time since last ad or number of ads already shown.

If moderation fails, the system returns a `no_fill` response and does not serve an ad.

3 CTR Estimation and Updating

Click-Through Rate (CTR) is a core metric in digital advertising, defined as the probability that a user clicks an ad given that it was shown. In our platform, CTR plays a central role in ad selection, auction ranking, advertiser and publisher billing, and publisher reporting.

3.1 CTR Definition and Bayesian Smoothing

A naive CTR estimate is given by the empirical ratio:

$$\widehat{CTR} = \frac{clicks}{impressions}.$$

However, this estimate is extremely unstable when impression counts are small, which is common in contextual LLM advertising.

For this reason, we use a **Bayesian Beta-Binomial framework**: CTR is modeled with a Beta prior:

$$CTR \sim Beta(\alpha_0, \beta_0),$$

where the prior mean is determined by global and contextual feature weights in logit space:

$$m_0 = \sigma(w_0 + w_{ad}[a] + w_{dom}[d] + w_{ctx}[c]).$$

After observing I impressions and C clicks for a given bucket (e.g., ad category or domain type), the posterior is:

$$CTR \mid C, I \sim Beta(\alpha_0 + C, \beta_0 + I - C),$$

and the updated CTR estimate is:

$$\widehat{CTR}_{post} = \frac{\alpha_0 + C}{\alpha_0 + \beta_0 + I}.$$

This smoothed estimate avoids wild fluctuations at low volume and naturally incorporates uncertainty via the posterior variance and credible interval.

3.2 CTR Updating and Stability Guards

CTR values are updated over time as new impressions and clicks arrive. The platform applies guardrails based on:

- posterior variance,
- confidence-interval width,
- minimum impression thresholds.

These ensure CTR is only considered “trusted” once enough evidence has accumulated. For unstable buckets, auctions use a conservative lower-bound CTR estimate.

Further Detail. The full mathematical framework for CTR modeling—including Bayesian priors, posterior computation, guard conditions, feature-weight updates in logit space, global baseline calibration, and the transition from batch updating to real-time streaming—is described in the accompanying documentation: *CTR Modeling for Contextual Advertising in LLM-based Systems: The Adstrack Framework*.

3.3 Real-Time CTR Updating via Streaming

The CTR framework described above is mathematically identical whether updated daily in batch mode or continuously in real time. In the real-time version, a streaming engine maintains rolling counts of impressions and clicks for each ad, domain type, and conversation category.

As events (impressions and clicks) stream into the system, the following occurs:

1. decayed counts (I_k, C_k) are updated incrementally,
2. posterior Beta parameters are recomputed for each bucket,
3. smoothed CTR values and stability indicators are refreshed,
4. updated CTR estimates are written to a fast store (e.g., Redis) for immediate use in ranking and auctions.

This allows the platform to react to changing conditions within seconds rather than hours, and ensures that ad selection and auction decisions are based on the most up-to-date performance signals.

4 Real-Time Processing with Flink and Spart

To ensure that fraud detection, moderation signals, and CTR estimates remain accurate at scale, the platform uses a lightweight real-time processing layer. All internal events—such as ad requests, ad responses, and clicks—are streamed into Kafka, where Apache Flink and Spark process them continuously.

They maintain up-to-date aggregates such as:

- impression and click counts for CTR estimation,
- short-term performance metrics for ads and publishers,
- indicators of traffic anomalies at the publisher or IP level.

From these rolling statistics, they compute **fraud risk scores**, **CTR updates**, and other quality signals, which are stored in a fast cache (e.g., Redis). When the Django ad-serving API receives a new request, it reads these real-time features immediately to make an informed and safe decision.

This design allows the system to react within seconds to changes in traffic or user behavior, rather than relying on slower batch computations.

5 System Architecture Overview

Figure 1 illustrates the high-level data flow of the ad-serving platform. The diagram summarizes how ad requests, safety checks, real-time processing, and advertiser demand interact to produce a safe, contextual ad response.

5.1 End-to-End Flow

When a publisher platform issues an **Ad Request**, it enters the backend service, which performs the initial synchronous decision stages:

1. **Fraud checks** using the most recent risk scores stored in Redis,
2. **Conversation moderation** to determine suitability for ad placement,
3. **Vector search** to retrieve relevant candidate ads,
4. **Auctioning** to select the winning ad.

If the request passes these gates, the platform returns an **Ad Response** along with instructions for ad placement.

5.2 Event Streaming Into Kafka

All operational events generated by the publisher and by the backend are streamed into Kafka topics for downstream analytics:

- **Ad Request Events** Sent to the *Fraud* and *Moderation* topics, where they help track traffic anomalies, conversational patterns, and source behaviors.
- **Ad Acknowledgement (Impression) Events** Represent ad impression confirmations sent back to the platform. In the diagram, the **double-line arrows** indicate that these events are routed to *both* the fraud/moderation topics and the CTR topic.

- **Click Events** Click interactions recorded by the publisher. Like impression events, these also flow into *both* the fraud/moderation topics and the CTR topic (shown visually by the double lines).

This distinction reflects the roles of each streaming pipeline:

- Fraud and moderation require signals from requests, impressions, and clicks.
- CTR computation requires only impressions and clicks, not requests.

5.3 Real-Time Processing with Flink

Apache Flink consumes the fraud and moderation topics to maintain rolling real-time aggregates and compute:

- fraud risk scores per publisher, platform, and traffic source,
- moderation-related behavioral and stability signals,

which are continuously written into Redis for synchronous use by the backend.

In parallel, Flink consumes the CTR topic to update:

- smoothed Beta-Binomial CTR estimates,
- per-ad, per-context and and per-domain performance statistics

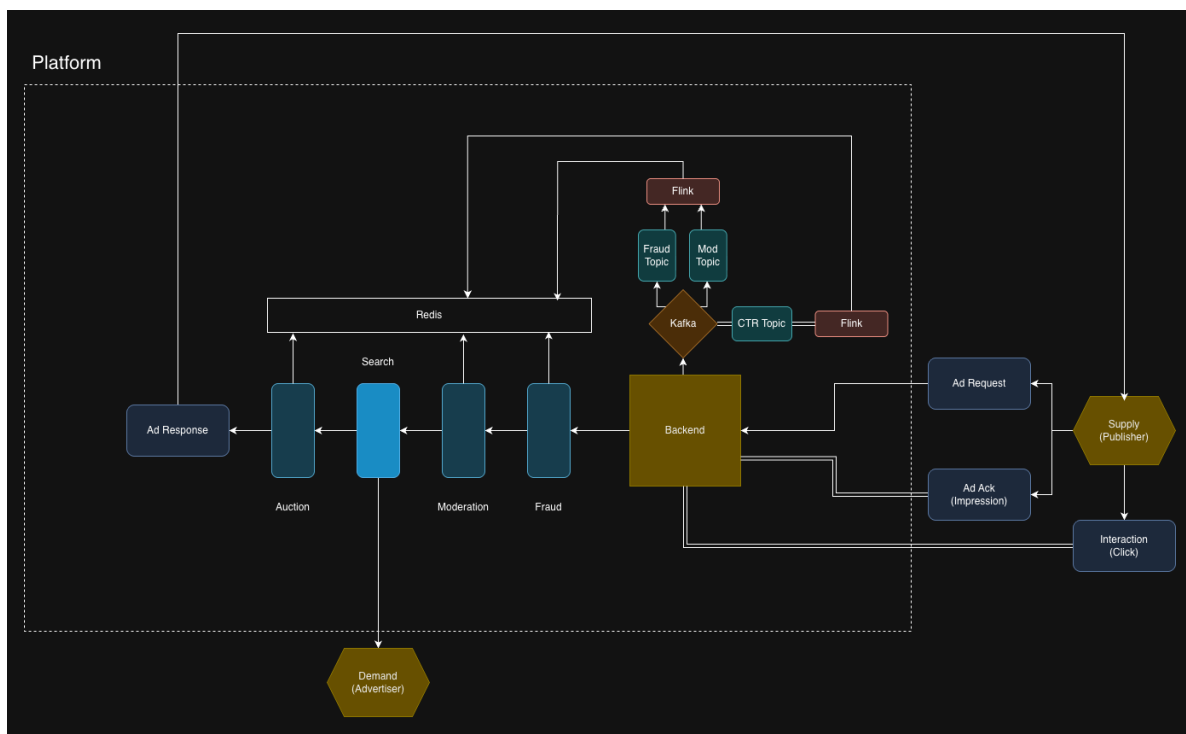


Figure 1: High-level architecture of the ad-serving pipeline, showing synchronous decision stages, real-time feature computation, and integration with advertiser demand and publisher supply.